- 📖 AgriScan Technical Documentation
  - 🏗️ System Architecture
    - 1. AI Inference Pipeline
    - 2. RAG Assistant Architecture
  - 💰 Financial Integration & Scoring
    - Scoring Algorithm Logic
    - Partner API Reference
  - 🔒 Security & RBAC
    - Role-Based Access Control (RBAC) Matrix
    - Infrastructure Security
  - 🚀 Mobile Native Integration
    - Key Integrations:
  - 🛠️ Setup & Installation
    - Essential Keys

📖 AgriScan Technical Documentation

Welcome to the **AgriScan** technical documentation. This document provides a deep dive into the system's architecture, security protocols, AI pipelines, and financial integration logic.

🏗️ System Architecture

AgriScan uses a decoupled modular architecture to handle real-time AI inference and complex data relationships.

1. AI Inference Pipeline

The system utilizes a multi-stage validation process for every crop scan:

- **Input Sanitization**: Images are renamed to UUIDs and validated for MIME-type (JPEG/PNG/WEBP) and size (5MB limit).
- **OOD Gatekeeper**: An **EfficientNetB0** model trained on ImageNet acts as a "Gatekeeper". It checks the image against a whitelist of ~80 plant-related classes

(leaves, flowers, trees, etc.). If the image doesn't match, it is rejected before reaching the crop models.
- **Model Routing**: Based on user selection, the system routes the image to a specific fine-tuned model (e.g., Maize-Net, Tomato-Net).
- **Result Logging**: Successful scans are logged in the `CropScanLog` table, which directly feeds into the Farm Health Score.

2. RAG Assistant Architecture

The **AgriBot** uses Retrieval-Augmented Generation to provide localized advice:

- **Knowledge Base**: PDF/Text documents are chunked and stored as embeddings in **ChromaDB**.
- **Retrieval**: User queries trigger a similarity search (K=3) using `all-MiniLM-L6-v2`.
- **LLM Synthesis**: The context is injected into a strict system prompt and processed by **Llama 3.3 (via Groq API)**.
- **Tool-Calling**: The bot can autonomously trigger the `get_weather_data` tool to fetch real-time forecasts for the user's county.

💰 Financial Integration & Scoring

AgriScan transforms digital footprints into a verifiable creditworthiness metric called the **Farm Health Score**.

Scoring Algorithm Logic

The score is calculated dynamically in `utils_scoring.py` and ranges from **300 to 850**:

Component	Max Points	Metric
Engagement	+200	10 pts per scan in the last 6 months.
Crop Health	+200	Ratio of "Healthy" diagnosis vs. disease detections.

Component	Max Points	Metric
Market Activity	+150	50 pts per completed transaction in the decentralized market.
Base Score	300	Automatic starting point for all registered farmers.

Partner API Reference

Financial institutions can retrieve a farmer's risk tier via a secure REST endpoint.

- **Endpoint**: `/api/farm-health-score/`
- **Method**: `GET`
- **Auth**: `Authorization: Bearer <API_KEY>`
- **Response**:

```
{
  "farmer": "job_nyaribari",
  "health_score": 745,
  "risk_tier": "Low Risk",
  "last_calculated": "2026-04-19T14:30:00Z"
}
```

🔒 Security & RBAC

Role-Based Access Control (RBAC) Matrix

Permissions are enforced at the view level using Django decorators and profile checks.

Feature	Farmer	Buyer	Agency
AI Crop Scanning	✅	❌	❌
AgriBot Access	✅	✅	✅
List Produce	✅	❌	❌
Manage Products	❌	❌	✅

Feature	Farmer	Buyer	Agency
Access Lending API	❌	❌	❌*

(API access is reserved for registered Partners with API Keys)

Infrastructure Security

- **Secure Storage**: In production, all media assets are stored on **Cloudinary** via a modern Django 5.x storage backend, ensuring persistence across ephemeral container restarts.
- **Rate Limiting**: Critical endpoints (Login, Signup, Chat, Scan) are protected by `django-ratelimit` to prevent brute-force and DDoS attacks.
- **Inactivity Protections**: Session timeouts are enforced after 30 minutes of inactivity.

🚀 Mobile Native Integration

AgriScan is packaged as a native mobile application using **CapacitorJS**.

Key Integrations:

- **Camera API**: Bypasses browser file pickers for a native camera interface.
- **Deep Linking**: Allows notifications to open specific disease results directly.
- **Resource Bundling**: Frontend assets are bundled locally to reduce load times on slow mobile networks.

🛠 Setup & Installation

Detailed setup instructions are available in the [README.md](file:///home/job/Desktop/projects/AgriScan/crop_AI_analyzer/README.md).

Essential Keys

- `SECRET_KEY`: Django security key.
- `GROQ_API_KEY`: For AgriBot LLM reasoning.
- `HF_TOKEN`: For model downloads and embeddings.
- `CLOUDINARY_URL`: For production media storage.

© 2026 AgriScan Development Team.